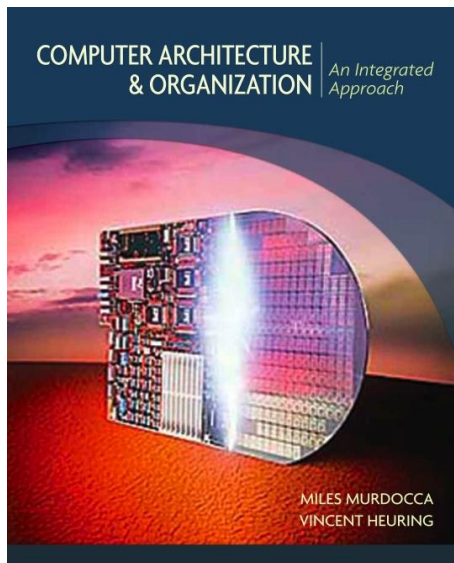


Organización de las Computadoras

Carolina Chaves Garcia



Timers

Introducción

Introducción

¿Qué son los Timers?

- Los timers (temporizadores) son **periféricos hardware** que funcionan como **"cronómetros programables"** dentro del microprocesador.
- Son **contadores que incrementan o decrementan su valor** automáticamente a una frecuencia conocida.
- Podemos decir que son como un "reloj de arena"
 - Los granos de arena = ciclos de reloj
 - El recipiente = registro del timer
 - Voltear el reloj = recarga del timer
 - Tiempo medido = retardos precisos



Introducción

Tipos de Timers en ESP32-C3

- El ESP32-C3 tiene dos grupos de timers (TIMG0 y TIMG1), cada uno con:
 - 2 timers de 64 bits (Timer0, Timer1)
 - 1 Watchdog Timer
- Timer Group 0 (TIMG0)
 - Timer 0 de 64 bits
 - Timer 1 de 64 bits
- Watchdog Timer
- Timer Group 1 (TIMG1)
 - Timer 0 de 64 bits
 - Timer 1 de 64 bits
- **Características Clave:**
 - 64 bits = Pueden contar hasta 2^{64} valores
 - Divisor programable = Pueden reducir la frecuencia de conteo
 - Auto-reload = Se recargan automáticamente al llegar a 0
 - Interrupciones = Pueden generar eventos al finalizar

Introducción

¿Para qué usamos los Timers?

- Retardos precisos Esperar 100ms para antirrebote
- Generación de señales PWM para controlar brillo LED
- Medición de tiempo Contar duración de pulsos
- Planificación Timeouts en comunicaciones

- **Delay por Software vs Hardware**

Aspecto	Software (bucles)	Hardware (Timer)
Precisión	Variable	Muy precisa
Uso de CPU	100%	Casi 0%
Flexibilidad	Limitada	Alta
Consumo	Alto	Bajo

Por qué Usamos Assembler para Esto

- Control directo sobre los registros del periférico
- Máxima eficiencia sin overhead del compilador
- Entendimiento profundo de cómo funciona el hardware
- Código determinístico sin optimizaciones inesperadas

Configuración de registros TIMG via MMIO

Configuración de registros TIMG via MMIO

Registros MMIO Clave del TIMG0

Registro	Dirección Base	Función
TIMG_T0CONFIG_REG	0x6001F000	Configuración Timer0
TIMG_T0LOADLO_REG	0x6001F018	Valor bajo de carga
TIMG_T0LOADHI_REG	0x6001F01C	Valor alto de carga
TIMG_T0LO_REG	0x6001F004	Valor bajo actual
TIMG_T0HI_REG	0x6001F008	Valor alto actual
TIMG_T0UPDATE_REG	0x6001F00C	Forzar actualización

TIMG_T0CONFIG_REG

Bits importantes:

Bit 31 (ENABLE): 1 = Timer habilitado, 0 = Timer deshabilitado

Bit 30 (INCREASE): 1 = Contador incremental, 0 = Contador decremental

Bit 29 (AUTORELOAD): 1 = Auto-recarga al llegar a 0

Bits [13:28] (DIVIDER): Divisor de clock (prescaler value)

00 = /2

01 = /16 ← Usaremos este

10 = /256

11 = /1

Nota: Revisando Table 3.3-3.

Module/Peripheral Address Mapping del manual técnico conocemos la dirección base del Timer Group 0 (TIMG0)

Antirrebote (Debounce) con Timer Hardware

Antirrebote (Debounce) con Timer Hardware

- **Problema:** Cuando presionamos un pulsador físico, los contactos metálicos no hacen conexión instantánea perfecta. Generan múltiples conexiones/desconexiones rápidas antes de estabilizarse.
- **Consecuencia:** El microprocesador ve múltiples pulsos en milisegundos en lugar de un solo pulso.
- **Datos del rebote:**
 - Duración típica: 1-50ms
 - Causa: Vibración mecánica de los contactos
 - Solución: Esperar que pase el período de rebote antes de leer el estado

Implementación con Timer Hardware

Vamos a crear un sistema completo que:

- Configura el timer (usando lo que ya aprendimos)
- Lee el pulsador con antirrebote
- Controla un LED al detectar pulsación válida
- Medición de performance con contador de ciclos

Ejemplos

Ejemplos

Encender un led

- El ESP32-C3 no usa el esquema de vectores rígidos del modelo estándar de RISC-V (CLINT) sino un sistema más rico:
 - El controlador de interrupciones permite hasta 31 interrupciones asíncronas
 - Cada interrupción puede configurarse con prioridad (niveles 1 a 15) y tipo (nivel o flanco).
 - Interrupción por nivel: La interrupción se mantiene activa mientras el nivel de la señal esté presente. Si la condición (nivel alto o bajo) persiste, la interrupción puede generarse continuamente. # Nivel bajo (0) - Se activa mientras GPIO = 0
 - Interrupciones por Flanco: La interrupción se activa solo en la transición (flanco de subida, bajada o ambos). Una vez detectado el flanco, la interrupción se marca incluso si la señal vuelve a su estado original. # Flanco de subida (0→1)

Ejemplos

Encender un led

- CONSTANTES - DIRECCIONES MMIO
 - Registros del Timer Group 0 ✓
 - Frecuencias y cálculos ✓
 - CPU_FREQ ✓
 - TIMER_DIVIDER ✓
 - TIMER_SCALE ✓
 - Inicialización completa del Timer Group 0
- PASO 1: CONFIGURAR REGISTRO T0CONFIG
 - Restaurar registros ✓
 - Generar retardo en ms ✓
 - Guardar parámetro ✓
 - Cargar dirección base del TIMG0 ✓
- PASO 2: CALCULAR TICKS PARA EL RETARDO
 - **Fórmula:** $\text{ticks} = (\text{ms} * \text{TIMER_SCALE}) / 1000$ ✓
- PASO 3: CARGAR VALORES EN REGISTROS LOAD
 - Separar valor 64-bit en partes alta y baja ✓
 - Escribir en registros de carga ✓
- PASO 4: FORZAR ACTUALIZACIÓN DEL TIMER ✓
- PASO 5: ESPERAR QUE TIMER LLEGUE A CERO ✓
- CONFIGURAR **GPIO3 (LED) COMO SALIDA**. Puede escoger otro GPIO si lo desea ✓

Ejemplos

Antirrebote

- Hay registros mapeados en memoria para habilitar/deshabilitar, limpiar, estado pendiente, etc.
- Por ejemplo, el registro INTERRUPT_CORE0_CPU_INT_ENABLE_REG (nombre genérico) controla el estado de habilitación de interrupciones por núcleo.
- El registro INTERRUPT_CORE0_CPU_INT_CLEAR_REG sirve para limpiar las interrupciones de tipo flanco (toggle) cuando han sido atendidas.
- **INTERRUPT_CORE0_SOURCE_X_MAP_REG**
- **INTERRUPT_CORE0_GPIO_INTERRUPT_PRO_NMI_MAP_REG**
- **INTERRUPT_CORE0_TG_T0_INT_MAP_REG**
- **INTERRUPT_CORE0_CPU_INT_ENABLE_REG**
- **INTERRUPT_CORE0_CPU_INT_CLEAR_REG**
- **INTERRUPT_CORE0_CPU_INT_TYPE_REG**
- **CPU_INT_ID**

Ejemplos

Antirrebote

- **CONSTANTES Y DIRECCIONES**

- GPIO2 para LED ✓
- TIMG0 para el timer ✓
- Tiempos de antirrebote: 50ms ✓

- **CONFIGURAR TIMER**

- **debounce_init:** Configuramos el timer en esta función para poder iniciar el tiempo de espera del antirebote ✓

- **timer_delay_ms:** Establecemos el delay (igual que hicimos en el anterior) ✓

- **debounce_read:** Leer pulsador con antirrebote

- Parámetro: a0 = número de GPIO a leer ✓
- Retorna: a0 = 1 si pulsación válida, 0 si no ✓
- PASO 1: Detectar flanco de bajada (presionado) ✓

- Leer estado actual del GPIO ✓

- Si está en 1 (no presionado), salir ✓

- PASO 2: Esperar período de antirrebote (50ms)

- Esperar 50ms usando timer hardware, llamando a timer_delay_ms ✓

- PASO 3: Verificar que sigue presionado después del rebote

- Leer GPIOs nuevamente. Leer registro de entrada GPIO ✓

- Si ya no está presionado, era rebote. Si es 1, no está presionado ✓

Ejemplos

Antirebote

- PASO 4: Esperar que se suelte el pulsador
 - Esperar hasta que sea 1 (suelto) ✓
 - PASO 5: Anti Rebote del release
 - Esperar 50ms después de soltar ✓
 - Pulsación válida detectada ✓
- No se detectó pulsación válida ✓
- debounce_end: Cargar todos los valores necesarios en memoria y retornar. ✓

Gracias